

オペレーティングシステム論 レポート 2

神野 友哉

2022311042

愛知県立大学 情報科学部 情報科学科

2024 年 1 月 9 日

1 はじめに

本レポートでは、食事をする哲学者問題の解法の一つである、Chandy / Misra 法のプログラムを作成し、各哲学者の初期状態を様々に変化させて実験する。

2 作成したプログラムについて

プログラム 1 を作成した。松尾啓志の「オペレーティングシステム (第 2 版) (情報工学レクチャーシリーズ)」P 62 の疑似コードを参考にした [1]。また、疑似コードを実際の c 言語に書き起こす際に、Oracle Help Center のウェブサイトを参考にした [2]。13 行目の `int state[N]=;` に宣言時代入することで、初期状態を調整した。

source 1: chandy / misra Algorithm

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5
#define THINKING 0
#define HUNGRY 1
#define EATING 2
#define TRUE 1

int state[N] = {EATING, EATING, EATING, EATING, EATING};
sem_t mutex;
sem_t s[N]={0};

void test(int i) {
    if (state[i] == HUNGRY && state[(i + N - 1) % N] != EATING && state[(i + N + 1) % N] != EATING) {
        state[i] = EATING;
        sem_post(&s[i]);
    }
}

void take_forks(int i) {
    sem_wait(&mutex);
    state[i] = HUNGRY;
    test(i);
    sem_post(&mutex);
    sem_wait(&s[i]);
}

void put_forks(int i) {
```

```

        sem_wait(&mutex);
        state[i] = THINKING;
        test((i + N - 1) % N);
        test((i + N + 1) % N);
        sem_post(&mutex);
    }

    void philosopher(int i) {
        while(TRUE) {
            take_forks(i);
            printf("Philosopher %d is eating.\n", i);
            put_forks(i);
        }
    }

    int main(void) {
        int sinit, i;
        pthread_t philosophers[N];
        sem_init(&mutex, 0, 1);

        for (i = 0; i < N; i++) {
            sinit = sem_init(&s[i], 0, 0);
        }

        for (i = 0; i < N; i++) {
            pthread_create(&philosophers[i], NULL, (void*)philosopher, (void*)(intptr_t)i);
        }

        for (i = 0; i < N; i++) {
            pthread_join(philosophers[i], NULL);
        }

        return 0;
    }
}

```

3 実験結果

全員の初期状態を思考中 (THINKING) にした場合で実行したところ、半日という長時間の試行でも、デッドロックせずに全哲学者が食事をしていった。しかし、動作の規則性等を確認したところ、多少の癖が見られた。隣り合う哲学者同士が交互に食事をする長期間と、定期的に規則性なく異なった哲学者が食事をする短期間が確認され、前者の隙間に後者が存在しているようであった。例として、哲学者 3、4 が交互に食事をし合った後に、哲学者 3, 1, 4, 0, 2, 4, 1, 0, 3, 1... のように食事をした後、また哲学者 0、3 が交互に食事をするといった様子である。また、全員の初期状態を食事中 (EATING) にした場合や、空腹中 (HUNGRY)、哲学者 2 のみ食事中 (EATING) でそれ以外は空腹中 (HUNGRY) とした場合も、それぞれ 10 分 試行したが同様にデッドロックしなかった。念のためそれぞれのシチュエーションで複数回試行している。

4 考察

長時間放置しても、すべての哲学者が食事を得られていることや、各初期状態でその目的が達成されていたことを考慮すると、Chandy / Misra 法のプログラムとして、作成できたと考えられる。また、この解法において、特定の二人の哲学者が交互に食事中宣言をする原因として、隣り合わない哲学者の二人が同時に食事をし合うか、隣り合う哲学者の二人がフォークを渡しあって交互に食事をしていると考えられる。また、プログラム自体は簡素であり、かつ、哲学者の人数等の拡張性や新しい状態を追加する余地も備えているため、コードとしてよいプログラムであると考えられる。

5 感想

正直なところ、初期状態が動作に与える影響を見つけようと、血眼になって PowerShell を観察したが、いまいち違いが判らなかったため、とても残念だった。集計プログラムを作ってみたが、やはりよく判らなかった為、敢えて書かない判断とした。

参考文献

- [1] 松尾啓志. 『オペレーティングシステム (第 2 版) (情報工学レクチャーシリーズ)』. 第二版. ページ 62. (参照 2024-1-15).
- [2] Oracle Help Center. ”セマフォ (マルチスレッドのプログラミング)”. Oracle Help Center. 2010.
<https://docs.oracle.com/cd/E19455-01/806-2732/6jbu8v6os/index.html>, (参照 2024-1-15).